

Nama: Muhammad Akmal Dwiansyah Putra

Kelas / Absen: TI_1G / 20

NIM: 254107020110

Percobaan Jobsheet 6

Percobaan 1:

Mahasiswa20.java:

```
package Jobsheet6;
```

```
public class Mahasiswa20 {
```

```
    String nim;
```

```
    String nama;
```

```
    String kelas;
```

```
    double ipk;
```

```
    Mahasiswa20(){
```

```
}
```

```
    Mahasiswa20(String nim, String nama, String kelas, double ipk){
```

```
        this.nim = nim;
```

```
        this.nama = nama;
```

```
        this.kelas = kelas;
```

```
        this.ipk = ipk;
```

```
}
```

```
    void TampilkanInformasi(){
```

```
        System.out.println("Nama   : " + nama);
```

```
        System.out.println("NIM    : " + nim);
```

```
        System.out.println("Kelas : " + kelas);
```

```
        System.out.println("IPK    : " + ipk);
```

```
}  
}
```

MahasiswaBerprestasi20.java:

```
package Jobsheet6;
```

```
public class MahasiswaBerprestasi20 {  
    Mahasiswa20[] listSiswa = new Mahasiswa20[5];
```

```
    int idx;
```

```
    void Tambah(Mahasiswa20 m){  
        if (idx < listSiswa.length) {  
            listSiswa[idx] = m;  
            idx++;  
        }else{  
            System.out.println("Data Sudah Penuh");  
        }  
    }  
}
```

```
    void Tampil(){  
        for (Mahasiswa20 m : listSiswa) {  
            m.TampilkanInformasi();  
            System.out.println("-----");  
        }  
    }  
}
```

```
    int SequentialSearch(double cari){  
        int posisi = -1;  
        for (int i = 0; i < listSiswa.length; i++) {  
            if (listSiswa[i].ipk == cari) {  
                posisi = i;  
                break;  
            }  
        }  
    }  
}
```

```

        }
    }
    return posisi;
}

void tampilPosisi(double x, int pos){
    if (pos != -1) {
        System.out.println("Data Mahasiswa dengan ipk " + x + " ditemukan pada index ke-" + pos);
    }else{
        System.out.println("Data " + x + " tidak ditemukan");
    }
}

void tampilDataSearch(double x, int pos){
    if (pos != -1) {
        listSiswa[pos].TampilkanInformasi();
    }else{
        System.out.println("Data Mahasiswa dengan IPK " + x + " tidak ditemukan");
    }
}
}

```

MahasiswaDemo20.java:

```

package Jobsheet6;

import java.util.Scanner;

public class MahasiswaDemo20 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        MahasiswaBerprestasi20 list = new MahasiswaBerprestasi20();
        int jumlah = 5;
    }
}

```

```

for (int i = 0; i < jumlah; i++) {
    System.out.println("Input Data Mahasiswa ke-" + (i+1) + ":");

    String nim, nama, kelas;
    double ipk;

    System.out.print("Masukkan NIM   :");
    nim = sc.nextLine();

    System.out.print("Masukkan Nama  :");
    nama = sc.nextLine();

    System.out.print("Masukkan Kelas  :");
    kelas = sc.nextLine();

    System.out.print("Masukkan IPK   :");
    ipk = sc.nextDouble();

    sc.nextLine();

    Mahasiswa20 mahasiswa20 = new Mahasiswa20(nim, nama, kelas, ipk);
    list.Tambah(mahasiswa20);
}

list.Tampil();

System.out.println("-----");
System.out.println("Pencarian Data");
System.out.println("-----");
System.out.print("Input IPK Mahasiswa yang ingin dicari: ");
double cari = sc.nextDouble();

```

```
System.out.println("Menggunakan Sequential Searching");  
double posisi = list.SequentialSearch(cari);  
int pss = (int)posisi;  
list.tampilPosisi(cari, pss);  
list.tampilDataSearch(cari, pss);  
sc.close();  
}  
}
```

HASIL:

```
Input Data Mahasiswa ke-1:  
Masukkan NIM      :1  
Masukkan Nama     :1  
Masukkan Kelas    :1  
Masukkan IPK      :3.9  
Input Data Mahasiswa ke-2:  
Masukkan NIM      :2  
Masukkan Nama     :2  
Masukkan Kelas    :2  
Masukkan IPK      :3.5  
Input Data Mahasiswa ke-3:  
Masukkan NIM      :3  
Masukkan Nama     :3  
Masukkan Kelas    :3  
Masukkan IPK      :3.3  
Input Data Mahasiswa ke-4:  
Masukkan NIM      :4  
Masukkan Nama     :4  
Masukkan Kelas    :4  
Masukkan IPK      :3.8  
Input Data Mahasiswa ke-5:  
Masukkan NIM      :5  
Masukkan Nama     :5  
Masukkan Kelas    :5  
Masukkan IPK      :3.6
```

```
Nama      : 1
NIM       : 1
Kelas    : 1
IPK       : 3.9
```

```
Nama      : 2
NIM       : 2
Kelas    : 2
IPK       : 3.5
```

```
Nama      : 3
NIM       : 3
Kelas    : 3
IPK       : 3.3
```

```
Nama      : 4
NIM       : 4
Kelas    : 4
IPK       : 3.8
```

```
Nama      : 5
NIM       : 5
Kelas    : 5
IPK       : 3.6
```

```
-----
Pencarian Data
```

```
Input IPK Mahasiswa yang ingin dicari: 3.5
```

```
Menggunakan Sequential Searching
```

```
Data Mahasiswa dengan ipk 3.5 ditemukan pada index ke-1
```

```
Nama      : 2
NIM       : 2
Kelas    : 2
IPK       : 3.5
```

6.2.3. Pertanyaan

1. Jelaskan perbedaan metod tampilDataSearch dan tampilPosisi pada class MahasiswaBerprestasi!

Jawaban: Kedua method pada umumnya memiliki hal yang sama, yaitu mengoutput sebuah hasil. Tetapi perbedaan dari keduanya adalah tampilDataSearch menampilkan semua data mahasiswa yang dicari dari nama hingga ipk sedangkan tampilPosisi hanya menampilkan letak ipk pada array mahasiswa.

2. Jelaskan fungsi break pada kode program di bawah ini!

```
if (listMhs[j].ipk==cari){  
    posisi=j;  
    break;  
}
```

Jawaban: Break pada potongan program tersebut berfungsi untuk memberhentikan program. Jika bilangan yang dicari telah ditemukan maka program akan memberhentikan looping karena tidak efisien jika dilanjutkan terus.

3. Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?

Jawaban: Fungsi dari pos dan index hasil pencarian adalah untuk mengetahui Lokasi atau letak data yang di inginkan, ini dapat digunakan untuk memanggil tampilan data atau jika ingin mengetahui letak data tersebut.

4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan.

Jawaban: Jika terdapat lebih dari 1 IPK yang sama, program hanya akan mengambil data dari index paling depan, hal ini dikarenakan program di berhentikan ketika telah menemukan nilai yang sama, jadi nilai yang terdapat pada index lain tidak dapat di output.

5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah break dihapus dari kode di atas?

Jawaban: Jika perintah break dihapus pada potongan program, maka program akan tetap berjalan meskipun telah menemukan data yang dicari. Hal ini dapat digunakan jika ingin mencari lebih dari 1 data yang sama, tetapi untuk sementara hanya dapat menampung/mengoutput 1 data saja.

Percobaan 2:

MahasiswaBerprestasi20.java:

```
int findBinarySearch(double cari, int left, int right){  
    int mid;  
    if (left <= right) {  
        mid = (left+right)/2;
```

```

        if (cari == listSiswa[mid].ipk) {
            return (mid);
        } else if (listSiswa[mid].ipk > cari) {
            return findBinarySearch(cari, left, mid-1);
        } else {
            return findBinarySearch(cari, mid+1, right);
        }
    }
    return -1;
}

```

MahasiswaDemo20.java:

```

System.out.println("-----");
    System.out.println("Menggunakan Binary Searching");
    System.out.println("-----");
    double posisi2 = list.findBinarySearch(cari, 0, jumlah-1);

    System.out.println("Menggunakan Binary Searching");
    int pss2 = (int)posisi2;
    list.tampilPosisi(cari, pss2);
    list.tampilDataSearch(cari, pss2);

```


HASIL:

Input Data Mahasiswa ke-1:

Masukkan NIM :111

Masukkan Nama :adi

Masukkan Kelas :2

Masukkan IPK :3.1

Input Data Mahasiswa ke-2:

Masukkan NIM :222

Masukkan Nama :ila

Masukkan Kelas :2

Masukkan IPK :3.2

Input Data Mahasiswa ke-3:

Masukkan NIM :333

Masukkan Nama :lia

Masukkan Kelas :2

Masukkan IPK :3.3

Input Data Mahasiswa ke-4:

Masukkan NIM :444

Masukkan Nama :susi

Masukkan Kelas :2

Masukkan IPK :3.5

Input Data Mahasiswa ke-5:

Masukkan NIM :555

Masukkan Nama :anita

Masukkan Kelas :2

Masukkan IPK :3.7

Masukkan IPK :3.7

Nama : adi

NIM : 111

Kelas : 2

IPK : 3.1

Nama : ila

NIM : 222

Kelas : 2

IPK : 3.2

Nama : lia

NIM : 333

Kelas : 2

IPK : 3.3

Nama : susi

NIM : 444

Kelas : 2

IPK : 3.5

Nama : anita

NIM : 555

Kelas : 2

IPK : 3.7

Pencarian Data

Input IPK Mahasiswa yang ingin dicari: 3.7

Menggunakan Binary Searching

Menggunakan Binary Searching

Data Mahasiswa dengan ipk 3.7 ditemukan pada index ke-4

Nama : anita

NIM : 555

Kelas : 2

IPK : 3.7

6.3.3. Pertanyaan

1. Tunjukkan pada kode program yang mana proses divide dijalankan!

Jawaban: $\text{mid} = (\text{left} + \text{right}) / 2;$

2. Tunjukkan pada kode program yang mana proses conquer dijalankan!

Jawaban:

```
if (cari == listSiswa[mid].ipk) {  
    return (mid);  
} else if (listSiswa[mid].ipk > cari) {  
    return findBinarySearch(cari, left, mid-1);  
} else {  
    return findBinarySearch(cari, mid+1, right);  
}
```

3. Apa fungsi left, right, dan mid?

Jawaban: Fungsi dari ketiga variable tersebut adalah untuk menentukan posisi awal dan posisi akhir dari divide and conquer, ini adalah fungsi dari left dan right. Sedangkan mid menentukan posisi Tengan dari array tersebut serta menentukan apakah melanjutkan ke array sebelah kiri atau kanan.

4. Jika data IPK yang dimasukkan tidak urut. Apakah program masih dapat berjalan? Mengapa demikian?

Jawaban: Jika data IPK tidak urut, program akan menganggap semua data tidak memiliki bilangan yang kita cari, hal ini dikarenakan meskipun program berjalan dengan normal, program tetap terpengaruh pada besar kecil suatu data dari array tersebut. jadi jika tidak urut maka program tidak akan bisa mencari bilangan.

5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program binary seach agar hasilnya sesuai

Jawaban: Tidak sesuai, hasi binary search adalah tidak menemukan data yang dicari. Program dapat berjalan dengan sempurna dengan hasil yang diinginkan dengan cara membalikkan listMhs[mid].ipk > cari dari lebih dari menjadi kurang dari.

6. Jelaskan bagaimana binary search menentukan bahwa data yang dicari tidak ditemukan di dalam array.

Jawaban: Binary search menentukan suatu data ditemukan jika program tidak melakukan pemanggilan pada dirinya sendiri. Looping pada Binary search adalah menggunakan rekursif, jadi jika tidak dipanggil, maka program akan berhenti dan mengoutput -1 atau dalam situasi ini adalah data tidak dapat ditemukan

7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.

Jawaban:

```
System.out.print("Masukkan Jumlah Mahasiswa yang Diinput: ");
```

```
int jumlah = sc.nextInt();
```